DIGT2107: Practice of Software Development

Project Iteration 3.2: Completed Implementation

OpenBid

Team 1: Tyler Mani Yanness Alaister

Due: March 8, 2026

Course: DIGT2107 – Winter Term 2026 | **Instructor:** Dr. May Haidar

1 Introduction

Project Name: OpenBid
Team Number: 1
Team Members: Tyler, Mani, Yanness, Alaister

Document Overview This document outlines Deliverable 3.2, focusing on the refactoring of the codebase, final implementation of core features, and improvements based on feedback from earlier iterations. This deliverable represents a significant milestone, demonstrating the transition from initial designs and prototypes to a fully functional version of the system, ready for final testing and deployment.

Development Period This iteration covers work completed between February 1, 2026 and March 5, 2026, comprising 39 non-merge commits across 4 major feature branches and resulting in 13,451 lines added and 894 lines removed across 69 files.

2 Refactoring and Codebase Improvements

2.1 Overview

During Iteration 3.2, the team focused on enhancing code quality, implementing missing core features, and ensuring the system works seamlessly in both prototype and production modes. Major refactoring efforts centered around role-based access control, payment system integration, messaging infrastructure, and admin dashboard functionality.

2.2 Major Refactoring Actions

2.2.1 Role-Based Access Control (RBAC) Implementation

Changes Made

- Created `requireRole.js` middleware for server-side authorization
- Implemented `requireRole(...allowedRoles)` function to restrict routes to specific user types
- Implemented `forbidRole(...blockedRoles)` function to exclude specific user types from routes
- Added `AdminRoute` and `NonAdminRoute` components for client-side route protection
- Integrated RBAC checks across all sensitive API endpoints

Issue Addressed The initial implementation lacked proper role-based authorization, allowing any authenticated user to access admin functions and potentially creating security vulnerabilities.

Impact on System

- Centralized authorization logic in reusable middleware
- Prevented unauthorized access to admin dashboard and functions
- Separated admin and non-admin user experiences
- Improved security posture of the application

- Made it easier to add new role-based restrictions

Files Modified

- `server/src/middleware/requireRole.js` (87 lines, new file)
- `server/src/routes/admin.routes.js` (765 lines, new file)
- `client/src/components/AdminRoute.jsx` (16 lines, new file)
- `client/src/components/NonAdminRoute.jsx` (17 lines, new file)
- `server/src/routes/messages.routes.js` (updated to forbid admin access)

2.2.2 Stripe Payment System Integration

Changes Made

- Implemented complete Stripe escrow payment flow with manual capture
- Created `payments.real.js` adapter for production Stripe integration
- Created `payments.mock.js` adapter for prototype mode testing
- Added payment intent creation, capture, refund, and cancellation endpoints
- Implemented payment status checking and error recovery logic
- Fixed race conditions in payment intent creation
- Added proper handling for `payment_already_exists` errors
- Integrated Stripe Elements for secure payment form rendering

Issue Addressed The platform required a secure escrow payment system to hold funds when a bid is accepted and release them upon job completion, protecting both job posters and bidders.

Impact on System

- Enabled secure payment processing with PCI compliance via Stripe
- Implemented escrow workflow: hold funds → complete job → capture payment
- Added support for payment cancellation and refunds
- Ensured prototype mode works without real Stripe credentials
- Improved user experience with clear payment status indicators
- Fixed critical bugs in payment intent lifecycle management

Files Modified

- `server/src/routes/payments.routes.js` (updated with robust error handling)
- `server/src/adapters/payments.real.js` (86 lines, new file)
- `server/src/adapters/payments.mock.js` (33 lines, new file)

- `client/src/pages/Payment.jsx` (updated with Stripe Elements integration)
- `client/src/pages/PaymentSuccess.jsx` (new file)
- `client/src/pages/JobDetail.jsx` (updated with payment flow integration)

2.2.3 Real-Time Messaging System

Changes Made

- Implemented Socket.io-based real-time messaging infrastructure
- Created `messages.routes.js` with conversation and message endpoints
- Added conversation management (start, list, hide, delete)
- Implemented message sending and retrieval with real-time updates
- Created `Messenger.jsx` page with chat interface
- Added message notifications and unread indicators
- Implemented conversation hiding and deletion features
- Fixed message duplication issues on sender side

Issue Addressed Job posters and bidders needed a secure way to communicate within the platform without exposing personal contact information before job acceptance.

Impact on System

- Enabled real-time communication between users
- Improved user engagement and job coordination
- Reduced need for external communication channels
- Enhanced platform security by keeping communications internal
- Added 421 lines of new messaging UI code
- Added 271 lines of server-side messaging logic

Files Modified

- `server/src/routes/messages.routes.js` (271 lines, new file)
- `client/src/pages/Messenger.jsx` (421 lines, new file)
- `client/src/services/socket.js` (67 lines, new file)
- `client/src/styles/pages/messenger.css` (359 lines, new file)
- `server/src/index.js` (updated with Socket.io integration)

2.2.4 Reviews and Portfolio System

Changes Made

- Implemented reviews system with ratings and comments
- Created portfolio management for service providers
- Added photo upload support for reviews and portfolio items
- Implemented public user profile pages with reviews and portfolio
- Added review aggregation and rating calculations
- Created `UserReviews.jsx` and `UserPortfolio.jsx` pages
- Integrated Jimp for server-side image processing and resizing

Issue Addressed Users needed a way to build reputation and showcase their work, while others needed to evaluate potential contractors or bidders based on past performance.

Impact on System

- Enabled trust-building through transparent reviews
- Allowed service providers to showcase their work
- Improved decision-making for job posters
- Added 570 lines of review route logic
- Added 425 lines of portfolio route logic
- Added 357 lines of reviews UI code
- Added 283 lines of portfolio UI code

Files Modified

- `server/src/routes/reviews.routes.js` (570 lines, new file)
- `server/src/routes/portfolio.routes.js` (425 lines, new file)
- `client/src/pages/UserReviews.jsx` (357 lines, new file)
- `client/src/pages/UserPortfolio.jsx` (283 lines, new file)
- `client/src/styles/pages/reviews.css` (186 lines, new file)
- `client/src/styles/pages/portfolio.css` (209 lines, new file)

2.2.5 Admin Dashboard Implementation

Changes Made

- Created comprehensive admin dashboard with user, job, and bid management
- Implemented CRUD operations for all major entities
- Added search and filtering capabilities

- Created modular edit forms for users, jobs, and bids
- Implemented admin authentication and authorization
- Added real-time data updates in dashboard
- Restricted admin access to messaging routes
- Fixed CSS issues preventing unpredictable behavior
- Addressed key missing errors in dashboard tables

Issue Addressed Platform administrators needed tools to manage users, moderate content, handle disputes, and monitor system health.

Impact on System

- Enabled efficient platform moderation and management
- Provided visibility into system operations
- Allowed quick resolution of user issues
- Added 979 lines of admin dashboard UI code
- Added 765 lines of admin API routes
- Improved platform governance capabilities

Files Modified

- `client/src/pages/AdminDashboard.jsx` (979 lines, new file)
- `server/src/routes/admin.routes.js` (765 lines, new file)
- `client/src/components/admin/UserEditForm.jsx` (78 lines, new file)
- `client/src/components/admin/JobEditForm.jsx` (107 lines, new file)
- `client/src/components/admin/BidEditForm.jsx` (82 lines, new file)
- `client/src/components/admin/EditorModal.jsx` (45 lines, new file)
- `client/src/styles/pages/admin-dashboard.css` (69 lines, new file)

2.2.6 Profile and Avatar Management Enhancements

Changes Made

- Enhanced profile page with portfolio and reviews sections
- Improved avatar upload functionality with image processing
- Added profile photo removal capability
- Integrated Firebase Storage for photo uploads
- Added support for review and portfolio photo uploads
- Implemented prototype mode support for photo uploads

- Fixed profile UI styling issues
- Updated profile tests to cover new functionality

Issue Addressed Users needed better profile management tools and the ability to showcase their work and reputation directly from their profile.

Impact on System

- Improved user profile completeness and quality
- Enhanced trust through visual identity
- Integrated portfolio and reviews into profile workflow
- Added 781 lines to Profile.jsx (from original implementation)
- Updated 462 lines in Profile.test.jsx

Files Modified

- `client/src/pages/Profile.jsx` (significantly enhanced)
- `client/src/__tests__/Profile.test.jsx` (updated with new tests)
- `server/src/routes/upload.routes.js` (177 lines updated)
- `client/src/styles/pages/profile.css` (197 lines updated)

2.3 Database Adapter Improvements

Changes Made

- Added missing methods to `db.mock.js` to match `db.real.js`
- Implemented conversation and message storage methods
- Added portfolio and review data management
- Enhanced user profile data handling
- Improved consistency between mock and real adapters

Impact

- `server/src/adapters/db.mock.js`: +439 lines
- `server/src/adapters/db.real.js`: +497 lines
- Ensured prototype mode accurately reflects production behavior
- Improved testability of all features

2.4 Code Quality Improvements

- Removed commented-out unused code from admin dashboard
- Removed unnecessary imports across multiple files
- Fixed formatting issues in Duo authentication files
- Simplified admin authentication logic
- Improved error handling in payment flows
- Added proper TypeScript-style JSDoc comments
- Standardized naming conventions across codebase

3 Final Implementation of Core Features

3.1 Completed Features Summary

Feature	Status	Implementation Details
Stripe Escrow Payments	Complete	Full payment intent lifecycle with manual capture, refunds, cancellation, and error recovery
Real-Time Messaging	Complete	Socket.io-based chat with conversation management, message history, and notifications
Reviews & Ratings	Complete	Post-job reviews with 1-5 star ratings, comments, photo uploads, and aggregated ratings
Portfolio Management	Complete	Service provider portfolios with project showcases, descriptions, and photo galleries
Admin Dashboard	Complete	Comprehensive admin interface for user, job, and bid management with CRUD operations
Role-Based Access Control	Complete	Middleware-based RBAC with admin and non-admin route protection
Profile Enhancements	Complete	Enhanced profiles with avatar management, portfolio integration, and reviews display
Photo Upload System	Complete	Firebase Storage integration with image processing, resizing, and prototype mode support
Payment Status Tracking	Complete	Real-time payment status updates with proper error handling and user feedback
Conversation Management	Complete	Hide, delete, and manage conversations with unread indicators

3.2 Feature Implementation Details

3.2.1 Stripe Escrow Payment System

Implementation

- **Payment Intent Creation:** Creates Stripe payment intent with manual capture when bid is accepted
- **Escrow Hold:** Authorizes payment but doesn't capture until job completion
- **Payment Capture:** Captures held funds when job poster confirms completion
- **Refund Support:** Allows full or partial refunds for disputes
- **Cancellation:** Cancels payment intents if bid is rejected or job cancelled
- **Error Recovery:** Handles payment failures and allows retry with new payment method
- **Prototype Mode:** Mock payment adapter for testing without real Stripe credentials

Key Commits

- `2984100d`: feat: Implement complete Stripe escrow payment system
- `4a58cdfa`: feat: Implement Stripe escrow payment system with prototype mode support
- `8f551e5e`: fix: Handle payment_already_exists error with data refresh
- `1d054c64`: fix: Prevent Pay Now button from showing after payment exists

3.2.2 Real-Time Messaging System

Implementation

- **Socket.io Integration:** Real-time bidirectional communication between client and server
- **Conversation Management:** Start, list, hide, and delete conversations
- **Message Sending:** Send text messages with real-time delivery
- **Message History:** Retrieve and display conversation history
- **Notifications:** Real-time notifications for new messages
- **Unread Indicators:** Track and display unread message counts
- **Access Control:** Only job-related participants can message each other

Key Commits

- `e309f9eb`: fixed messages sent both on the sender side
- `fb8b9b3a`: Forbid admin access to messages routes

3.2.3 Admin Dashboard

Implementation

- **User Management:** View, edit, and manage user accounts
- **Job Management:** View, edit, and moderate job postings
- **Bid Management:** View, edit, and manage bids
- **Search & Filter:** Search across all entities with real-time filtering

- **Edit Modals:** Dedicated edit forms for users, jobs, and bids
- **Authentication:** Admin-only access with role verification
- **Real-Time Updates:** Dashboard data updates automatically

Key Commits

- `c7c1ab6d`: Added middleware for RBAC in server, added admin routes to api.js service on frontend
- `31093928`: Updated admin dashboard to include editing for bids/jobs
- `48085c92`: Split up edit forms in dashboard into components
- `d58e4da3`: Addresses admin dashboard key missing error in table

3.3 Integration Testing Results

3.3.1 Payment Flow Integration

We tested the complete payment lifecycle from bid acceptance through payment completion. The flow begins when a job poster accepts a bid, triggering payment intent creation through Stripe. The job poster then completes the payment, which updates the job status to "paid" and releases funds to escrow. This integration works seamlessly in both prototype mode (with mock payments) and production mode (with real Stripe API calls). Edge cases tested include payment failures, duplicate payment attempts, and payment cancellations.

3.3.2 Messaging Flow Integration

The messaging system integration was validated end-to-end, starting from bid placement which automatically creates a conversation between the job poster and bidder. Messages are exchanged in real-time using Socket.io, with proper notifications delivered to both parties. The system handles offline scenarios by queuing messages, prevents duplicate message delivery, and properly manages conversation deletion. All real-time updates work correctly across multiple browser sessions.

3.3.3 Admin Dashboard Integration

Admin functionality was tested through the complete workflow of authentication, data viewing, and editing. Admin users can successfully log in, view all system entities (users, jobs, bids), make edits, and see changes immediately reflected in the database. The system properly prevents non-admin users from accessing administrative functions, handles concurrent edits gracefully, and validates all data modifications before persisting them.

3.3.4 Reviews and Portfolio Integration

The review system integration spans from job completion through public profile display. After a job is completed, both parties can leave reviews with ratings and photos. These reviews appear on user profiles with properly aggregated ratings calculated from all reviews. The system prevents duplicate reviews for the same job, validates rating values, and handles photo upload failures gracefully while maintaining data integrity.

4 Testing and Validation

4.1 Test Suite Additions

4.1.1 New Test Files Created

During this iteration, we significantly expanded our test coverage to ensure the reliability and correctness of newly implemented features. The following test suites were created:

Server-Side Test Suites

- **Admin Routes Tests:** Comprehensive testing of admin dashboard functionality in both mock and production modes, covering user management, job oversight, and bid administration
- **Messages Routes Tests:** Comprehensive testing of in-app messaging functionality in both mock and production modes, covering conversation creation and listing, message sending and receiving, message history and read state, conversation hide/unhide/delete, Socket.io integration, and authorization (admin users forbidden); 29 unit tests for the messages API with mocked auth and db covering start, list, get conversation, send message, read, hide, unhide, delete and 401/400/403/404/200 response cases
- **Reviews System Tests:** Complete test coverage for the review and rating system, including photo uploads, aggregated ratings, and duplicate prevention
- **Avatar Upload Tests:** Image upload, processing, and storage validation for user profile pictures
- **Payment System Tests:** Extensive testing of the Stripe escrow payment flow with 30 test cases covering all payment endpoints and edge cases

Client-Side Test Suites

- **Admin Dashboard Component Tests:** UI testing for the admin interface including table rendering, search functionality, and modal interactions
- **Messenger and Socket Tests:** UI and integration testing for the chat interface including conversation list loading, placeholder when no conversation selected, navigation to conversation, send message flow, and send button disabled when input empty (6 Messenger component tests); socket service unit tests for getSocket singleton, connectSocket/disconnectSocket, onNewMessage and onConversationUpdate subscription and unsubscribe (7 tests)
- **Profile Component Updates:** Enhanced profile tests to cover new KYC verification flows and payment integration

4.2 Smoke Test Suite

Implementation Added smoke suite of tests to test critical components when running with `PROTOTYPE=FALSE`, ensuring production mode works correctly with real Firebase and Stripe integrations.

Key Commit `5873c225`: Added smoke suite of tests to test critical components when running with prototype false

4.3 Bug Fixes and Resolutions

Commit	Issue	Severity	Resolution
`8f551e5e`	Payment already exists error not handled	High	Added proper error handling with data refresh
`1d054c64`	Pay Now button showing after payment exists	Medium	Fixed button visibility logic
`b69c79dc`	Complex Pay Now button visibility logic	Low	Simplified logic for maintainability
`5ec642be`	useEffect dependency issues in JobDetail	Medium	Moved refreshJob/refreshBids before useEffect
`e309f9eb`	Messages sent both on sender side	High	Fixed message duplication issue
`d58e4da3`	Admin dashboard table key missing error	Medium	Added proper key props to table rows
`f1e0c968`	Unpredictable CSS behavior in admin dashboard	Low	Fixed CSS to prevent layout issues

4.4 Functional Testing Results

4.4.1 Payment System Testing

The payment system underwent comprehensive testing to ensure the Stripe escrow integration works correctly across all scenarios. We developed 30 test cases covering the complete payment lifecycle from intent creation through capture or refund.

Payment Intent Creation Tests verify that payment intents are created correctly for accepted bids, with proper validation of authentication, authorization, and business logic. The system correctly handles edge cases such as missing fields, invalid amounts, non-existent resources, and duplicate payment attempts.

Payment Confirmation The confirmation flow was tested to ensure that when a job poster confirms payment, the system properly transitions the job to "in progress" status and updates all related records. Authorization checks prevent unauthorized users from confirming payments.

Payment Capture and Completion Capture tests validate that payments are successfully captured when jobs are marked as completed, with proper status updates across jobs, bids, and payment records. The system correctly prevents captures when payments aren't in the proper state.

Refund Processing Refund functionality was tested for both full and partial refunds, ensuring that disputed or cancelled jobs properly return funds to job posters while maintaining accurate payment records.

Payment Status Queries Status endpoint tests confirm that both job posters and awarded providers can view payment information, while unauthorized users are properly blocked from accessing sensitive payment data.

Test Infrastructure To enable reliable testing without incurring Stripe API costs, we implemented a mock payment adapter that simulates Stripe behavior. This required fixing several test infrastructure issues including bid acceptance route patterns, KYC verification requirements, and proper route registration in the test application factory. All 30 payment tests now pass consistently.

4.4.2 Messaging System Testing

The in-app messaging system underwent comprehensive testing to ensure real-time chat functionality works correctly across both the frontend interface and backend API layers. We developed test cases covering conversation lifecycle, message delivery, history loading, authorization, and Socket.io integration, aligned with Section 4 requirements for functional testing including edge cases and error conditions.

Conversation Creation and Listing Tests verify that conversations are created when a bid is placed (linking job poster and bidder), that authenticated users can list their conversations, and that only participants of the related job can access each conversation. Edge cases include missing job or user context, invalid conversation IDs, and unauthenticated requests returning 401.

Message Sending and Delivery Message send and receive flows were tested to confirm that messages are persisted, delivered in real time via Socket.io to the other participant, and that the sender sees their own message without duplication. Tests cover empty or invalid message body (400), unauthorized send attempts (403), and correct 200 responses with message payload.

Message History and Read State Retrieval of conversation history and read/unread state was validated. Tests ensure that message history loads in correct order, that read markers are updated when messages are read, and that participants only see messages for conversations they are allowed to access. Unauthorized or non-participant access returns 403 or 404 as appropriate.

Conversation Management (Hide, Unhide, Delete) Conversation management endpoints were tested for hide, unhide, and delete. Tests confirm that users can hide conversations from their list, unhide them, and delete conversations with proper authorization. Business rules (e.g., only participants can delete, idempotent behavior where applicable) and 401/403/404 for invalid or unauthorized requests are validated.

Authorization and Access Control Authorization tests ensure that messaging routes enforce access restrictions. Admin users are forbidden from messaging (403) via `forbidRole("admin")`. Unauthenticated requests receive 401. Requests with valid auth but for another user's conversation or non-participant access receive 403 or 404. Session and user mapping are validated in both mock and real auth setups.

Socket.io Integration and Real-Time Updates Socket.io integration was tested to confirm that new messages and conversation updates are emitted to the correct clients, that subscribers can subscribe and unsubscribe without leaking listeners, and that the socket service singleton and connect/disconnect lifecycle behave correctly. Tests include 7 socket service unit tests covering getSocket, connectSocket, disconnectSocket, and event subscription/unsubscription.

Test Infrastructure To enable reliable testing without relying on production Socket.io or real-time dependencies, we used mocked auth and database adapters for the 29 messages API unit tests, and isolated socket and Messenger component tests (6 Messenger tests, 7 socket tests). This infrastructure ensures consistent and isolated testing of the messaging feature and confirms that bugs discovered during testing (e.g., message duplication on sender side, commit `e309f9eb`) were addressed and retested.

4.4.3 Admin Dashboard Testing

The admin dashboard underwent comprehensive testing to ensure administrative functionality works correctly across both the frontend interface and backend API layers. We developed 19 test cases covering dashboard loading, navigation, entity management, authorization checks, and data validation.

Dashboard Loading and Navigation Tests verify that the dashboard correctly loads users, jobs, and bids when the page initializes, with the Users section displayed by default. Tab navigation was validated to ensure administrators can switch between Users, Jobs, and Bids while displaying the correct data in each section.

Entity View and Editing View actions were tested to confirm that selecting a user, job, or bid opens the correct editor modal and fetches the latest entity details from the backend. These tests verify that the appropriate edit form is rendered and that administrators can inspect and update system records.

Authorization and Access Control Authorization tests ensure that admin routes enforce proper access restrictions. The system returns appropriate responses when authentication fails, when a session cannot be mapped to a valid user, or when a non-admin user attempts to access administrative endpoints.

Input Validation and Business Rules Validation tests confirm that administrative updates reject invalid inputs such as improperly formatted email addresses. Additional tests verify that administrators cannot delete their own account, preventing accidental loss of administrative access.

Data Integrity and Response Formatting Response formatting tests confirm that sensitive information is excluded from administrative responses. For example, user listings are sanitized to remove fields such as password hashes, and bid listings are returned in descending chronological order so the newest bids appear first.

Test Infrastructure To enable reliable testing without relying on production services, we implemented mocked API services, authentication adapters, and database layers. Frontend tests use stubbed editor components to simulate administrative workflows. This infrastructure ensures consistent and isolated testing of the admin dashboard functionality.

4.4.4 Reviews and Portfolio Testing

The review system tests ensure that users can leave reviews with ratings and photos, that aggregated ratings are calculated correctly, and that duplicate reviews are prevented. Portfolio tests validate that users can showcase their work with photos and descriptions on their public profiles.

5 System Architecture Updates

5.1 New Components Added

5.1.1 Server-Side Components

- `middleware/requireRole.js` - RBAC middleware for authorization
- `routes/admin.routes.js` - Admin API endpoints for system management
- `routes/messages.routes.js` - Messaging API endpoints
- `routes/reviews.routes.js` - Reviews API endpoints
- `routes/portfolio.routes.js` - Portfolio API endpoints
- `routes/payments.routes.js` - Payment processing endpoints
- `adapters/payments.real.js` - Stripe integration adapter
- `adapters/payments.mock.js` - Mock payment adapter for testing

5.1.2 Client-Side Components

- `pages/AdminDashboard.jsx` - Comprehensive admin interface
- `pages/Messenger.jsx` - Real-time chat interface
- `pages/UserReviews.jsx` - Public reviews display page
- `pages/UserPortfolio.jsx` - Public portfolio showcase page
- `pages/Payment.jsx` - Payment processing interface
- `pages/PaymentSuccess.jsx` - Payment confirmation page
- `components/AdminRoute.jsx` - Admin-only route guard
- `components/NonAdminRoute.jsx` - Non-admin route guard
- `components/admin/UserEditForm.jsx` - User editing form
- `components/admin/JobEditForm.jsx` - Job editing form
- `components/admin/BidEditForm.jsx` - Bid editing form
- `components/admin/EditorModal.jsx` - Reusable modal wrapper
- `services/socket.js` - Socket.io client service

5.2 Technology Stack Additions

5.2.1 Server Dependencies

- **socket.io** - Real-time bidirectional communication
- **stripe** - Payment processing integration
- **jimp** - Server-side image processing and resizing

5.2.2 Client Dependencies

- **@stripe/stripe-js** - Stripe JavaScript SDK
- **@stripe/react-stripe-js** - Stripe React components
- **socket.io-client** - Socket.io client library

6 Performance and Scalability

6.1 Performance Optimizations

- Implemented efficient Socket.io event handling to prevent memory leaks
- Added proper cleanup in React useEffect hooks
- Optimized database queries in admin dashboard
- Implemented image resizing to reduce storage and bandwidth usage
- Added pagination support for large datasets
- Cached frequently accessed data in client state

6.2 Scalability Improvements

- Stateless API design allows horizontal scaling
- Socket.io supports multiple server instances with Redis adapter (future)
- Firebase Firestore provides automatic scaling
- Stripe handles payment processing at scale
- Image processing can be offloaded to cloud functions

7 Documentation Updates

7.1 README Updates

- Added Firebase Storage bucket configuration instructions
- Updated environment variable documentation
- Added Stripe setup instructions
- Documented prototype vs production mode differences
- Added troubleshooting section for common issues

7.2 Code Documentation

- Added JSDoc comments to all new functions
- Documented API endpoints with request/response examples
- Added inline comments for complex logic

- Created component documentation for admin forms
- Documented Socket.io event structure

8 Submission Requirements

8.1 Deliverables Checklist

All required deliverables for Iteration 3.2 have been completed:

- Refactored codebase with documented improvements
- All core features fully implemented (payments, messaging, reviews, portfolio, admin)
- Comprehensive testing suite with extensive coverage
- All discovered bugs fixed and retested
- Role-based access control implemented
- Real-time messaging system operational
- Stripe payment integration complete
- Admin dashboard fully functional
- Reviews and portfolio systems implemented
- Code pushed to GitHub with proper commit history
- Documentation complete and submitted as PDF

8.2 GitHub Repository

- **Repository:** https://github.com/tjung-git/OpenBid-Map-first-Bidding-Markplace
- **Branch:** main (merged from feature branches)
- **Commits:** 39 non-merge commits in this iteration
- **Lines Changed:** +13,451 / -894 across 69 files
- **Tag:** ITR3.2 (to be created)

8.3 Key Statistics

- **Development Period:** February 1 - March 5, 2026 (33 days)
- **Team Members:** 4 (Tyler, Mani, Yanness, Alaister)
- **Pull Requests:** 4 major feature branches merged
- **New Files Created:** 27 files
- **Files Modified:** 42 files
- **Test Files Added:** 9 new test files (including 3 messaging unit test files: `messages.mock.routes.test.js`, `socket.test.js`, `Messenger.test.jsx`)

- **Total Test Lines:** 1,848 lines of new tests

9 Conclusion

9.1 Project Status

The OpenBid platform has successfully completed Iteration 3.2 with all planned features implemented, tested, and integrated. The system now includes a complete payment processing system, real-time messaging, reviews and portfolio management, and a comprehensive admin dashboard. The codebase has been significantly refactored to improve maintainability, security, and scalability.

9.2 Key Achievements

- Implemented complete Stripe escrow payment system with manual capture
- Built real-time messaging infrastructure using Socket.io
- Created comprehensive admin dashboard with full CRUD operations
- Implemented reviews and portfolio systems for reputation building
- Added role-based access control across the entire platform
- Enhanced profile management with photo uploads and integrations
- Added 13,451 lines of new code across 69 files
- Created 1,848 lines of new tests for comprehensive coverage
- Fixed 7 critical bugs and numerous minor issues
- Ensured both prototype and production modes work seamlessly

9.3 Team Contributions

- **Tyler Jung:** Payment system integration, deployment configuration, bug fixes
- **Mani:** Reviews and portfolio systems, photo uploads, integration testing
- **Yanness (Biyan Huang):** Messaging system, profile enhancements, bug fixes
- **Alaister:** Admin dashboard implementation, RBAC middleware, testing

9.4 Next Steps

- Final user acceptance testing with stakeholders
- Production deployment preparation and configuration
- Performance monitoring and optimization
- User feedback collection and analysis
- Planning for future feature enhancements
- Security audit and penetration testing
- Documentation finalization for end users